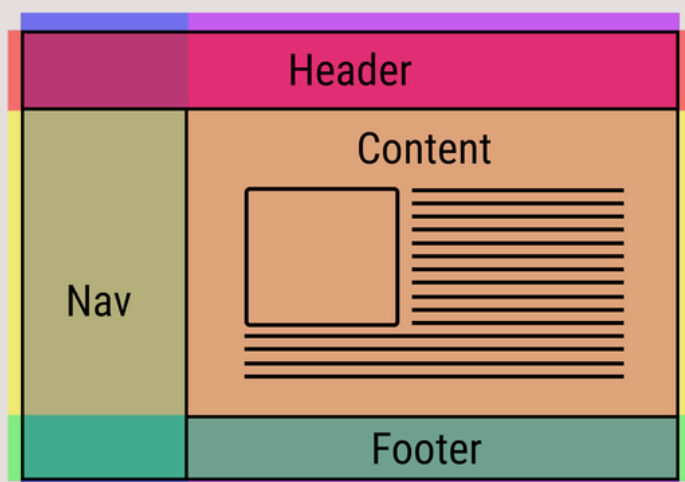


MATERIAL DE APOIO

CSS GRID LAYOUT

O CSS Grid Layout é uma ferramenta poderosa que revolucionou a forma como criamos layouts na web. Ele nos permite organizar elementos em uma grade bidimensional, oferecendo um controle preciso sobre o posicionamento e o tamanho dos elementos.

Imagine uma planilha. Ela é dividida em linhas e colunas, certo? O CSS Grid Layout funciona de forma similar mas aplicado ao contexto de criação de layouts.



(Disponível em: [Simple Web Layout with CSS Grid](#))

O CSS sempre foi usado para fazer o layout de nossas páginas da web, mas nunca fez um trabalho muito bom. Primeiro, usamos tabelas, depois floats, posicionamento e bloco embutido, mas todos esses métodos eram essencialmente “improvisos do desenvolvedor” e deixavam de fora muitas funcionalidades importantes (centralização vertical, por exemplo). O Flexbox também é uma ótima ferramenta de layout, mas seu fluxo unidirecional tem diferentes casos de uso - e eles realmente funcionam muito bem juntos! Grid é o primeiro módulo CSS criado especificamente para resolver os problemas de layout que todos nós temos “improvisado” desde que criamos sites.



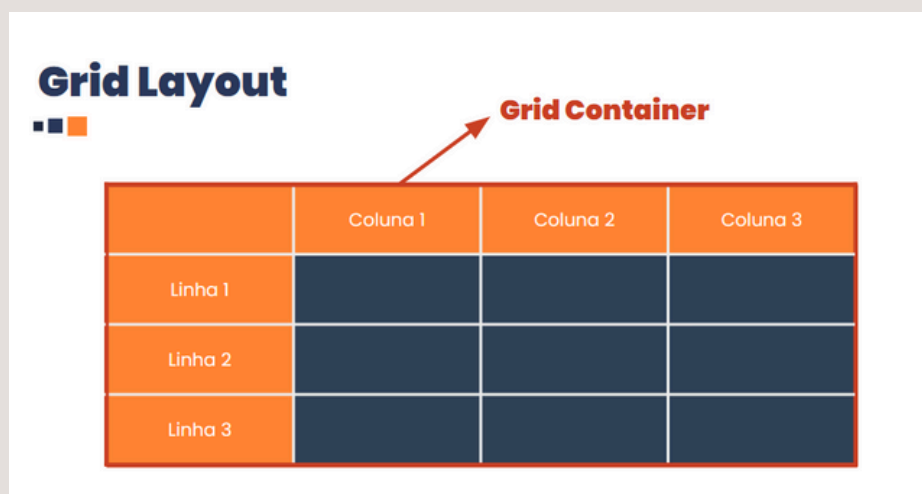
CONCEITOS BÁSICOS SOBRE GRID LAYOUT

Assim como o flexbox, o grid é um módulo e não uma única propriedade. Ele envolve muitas coisas, incluindo todo o seu conjunto de propriedades. Algumas dessas propriedades devem ser definidas/aplicadas no contêiner (elemento pai, conhecido como grid container), enquanto as demais devem ser definidas nos elementos filhos (chamados de grid itens) que são os itens da grade.

Antes de qualquer coisa é necessário entender a diferença entre as partes que compõem o Grid Layout.

- **Container:** o elemento que será a grade
- **Linhas:** são localizadas na horizontal da grade
- **Colunas:** são localizadas na vertical da grade

Na imagem abaixo é possível identificar a diferença entre elas:



ESTRUTURA BÁSICA

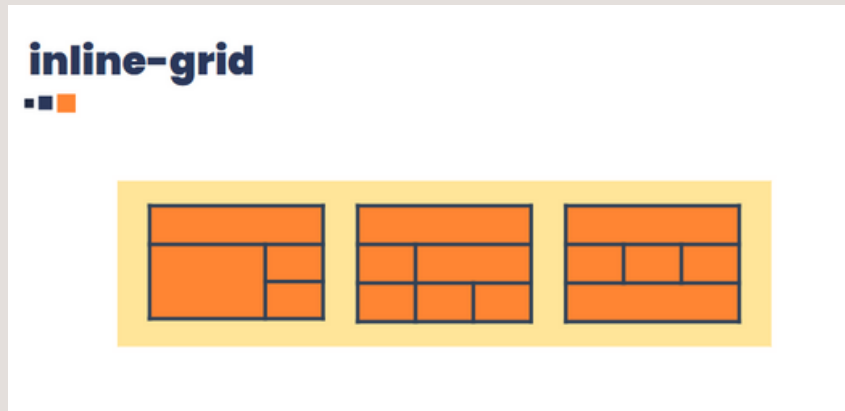
Para iniciarmos com o Grid Layout, a primeira coisa que precisamos fazer é definir a propriedade display no nosso container. Existem dois tipos:

display: grid e **display: inline-grid**.



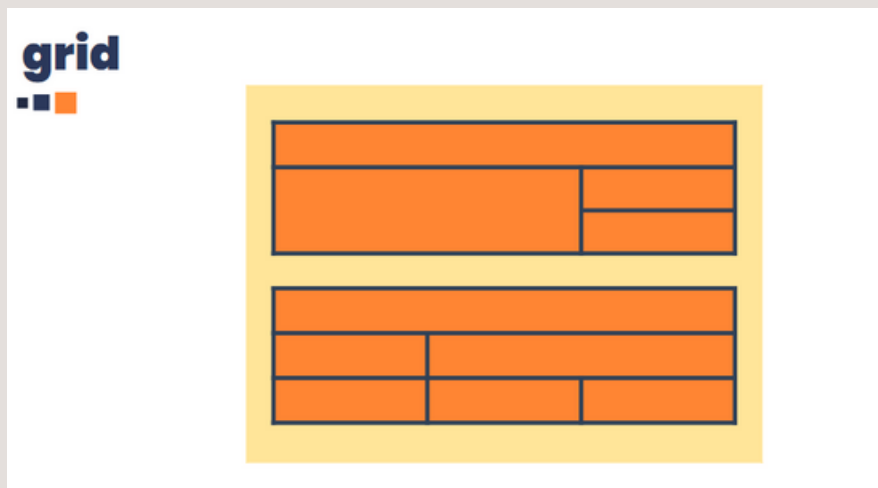
DISPLAY: INLINE-GRID

Nesta variação os grids são dispostos lado a lado, mesclando o comportamento inline com o comportamento grid.



DISPLAY: GRID

Nesta variação os grids são dispostos um abaixo do outro, mesclando o comportamento block com o comportamento grid.





CONSTRUINDO O NOSSO PRIMEIRO GRID LAYOUT

A partir de agora, iremos montar um grid layout e cada um dos exemplos será o complemento dos exemplos anteriores. Vamos começar!

A primeira coisa a se fazer é definir um container e setar a propriedade display para inline-grid ou grid. Neste exemplo, iremos utilizar o display: grid.

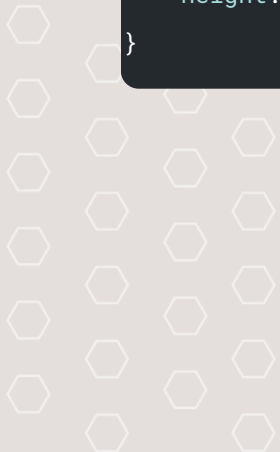
HTML

```
<section class="container">  
  
</section>
```

CSS

```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}
```

```
.container {  
  display: grid;  
  width: 100vw;  
  height: 100vh;  
}
```



DICA 💡

Neste momento você pode achar mais útil trabalhar no Firefox Developer Edition, que possui o [Grid Inspector](#) disponível como parte das Ferramentas do desenvolvedor.

Se você ver este exemplo no Firefox e inspecionar o Grid, você verá um pequeno ícone ao lado do valor Grid (na propriedade display do elemento container). Clique nele e o Grid neste elemento será sobreposto na janela do navegador.

À medida que você aprende e depois trabalha com CSS Grid Layout, esta ferramenta lhe dará uma ideia melhor do que está acontecendo com suas Grids visualmente.

GRID TEMPLATE ROWS E GRID TEMPLATE COLUMNS

Existem duas propriedades que geralmente vem acompanhadas com o display grid, são elas: grid-template-rows e grid-template-column. É dentro delas que definimos a quantidade de linhas e colunas que o nosso grid deve ter.

No código abaixo, iremos definir que o grid terá 3 linhas sendo que a primeira terá 100px de largura, a segunda pegará a largura automática e a terceira 50px.

O número de colunas será 2 sendo que a primeira coluna terá 300px e a segunda pegará o tamanho automático.

```
.container {
  display: grid;
  grid-template-rows: 100px auto 50px; /* código adicionado */
  grid-template-columns: 300px auto; /* código adicionado */
  width: 100vw;
  height: 100vh;
}
```

E agora para começar a dar vida ao nosso site, vamos criar um header.

```
<section class="container">
  <header></header>
</section>

/* ... */

header {
  background-color: cadetblue;
  grid-row: 1;
  /*
    grid-row-start: 1;
    grid-row-end: 1;
  */

  grid-column: 1/3;
  /*
    grid-column-start: 1;
    grid-column-end: 3;
  */
}
```

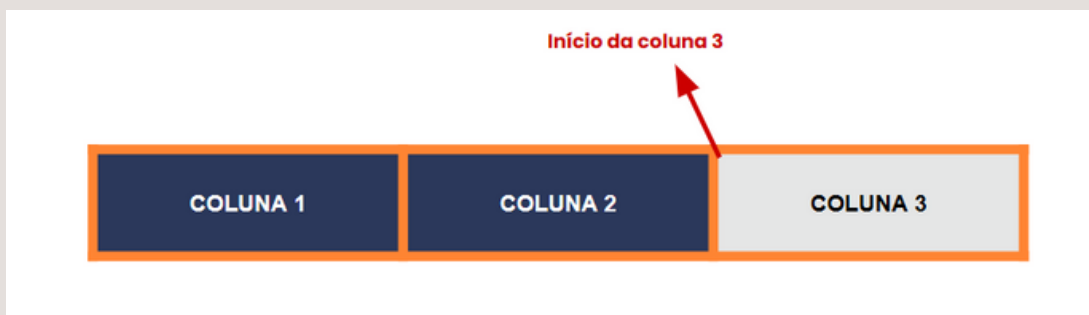


O código comentado simboliza uma outra maneira de definir, mas neste exemplo estaremos usando a maneira simplificada.

No CSS acima definimos uma cor para nosso header, além disso usamos a propriedade `grid-row` para informar em qual linha nós queremos que ele fique localizado (no exemplo, a primeira). Definimos também a propriedade `grid-column` para dizer quais colunas esse header deve ocupar (no caso ele vai começar na primeira coluna e terminar no início da terceira coluna).

Um pouco estranho, certo? Por que não definimos 3 colunas na propriedade `grid-template-columns` do container. A explicação é que, para informarmos que o header irá ocupar a coluna 1 e coluna 2, como dito anteriormente, indicamos o término no **INÍCIO** da terceira coluna.

Para exemplificar visualmente, deixamos abaixo um esquema bem simples de como isso funciona.



E o resultado é esse (figura abaixo): o header na primeira linha do grid ocupando todas as colunas.





Confira a seguir um exemplo da propriedade `justify-content` definida como `flex-start`. Existem também duas palavras-chave adicionais que você pode associar a esses valores: `safe` e `unsafe`. Usar o `safe` garante que, independentemente de como você faça esse tipo de posicionamento, você não pode empurrar um elemento para que ele fique fora da tela (por exemplo, fora do topo) de forma que o conteúdo também não possa ser rolado (chamado de "perda de dados").

Agora vamos criar um menu utilizando o elemento `nav`. Vamos indicar pelo CSS que o `nav` (nosso menu de navegação) vai estar localizado na segunda linha do grid e na primeira coluna.

HTML

```
<section class="container">
  <header></header>
  <nav></nav>
</section>
```

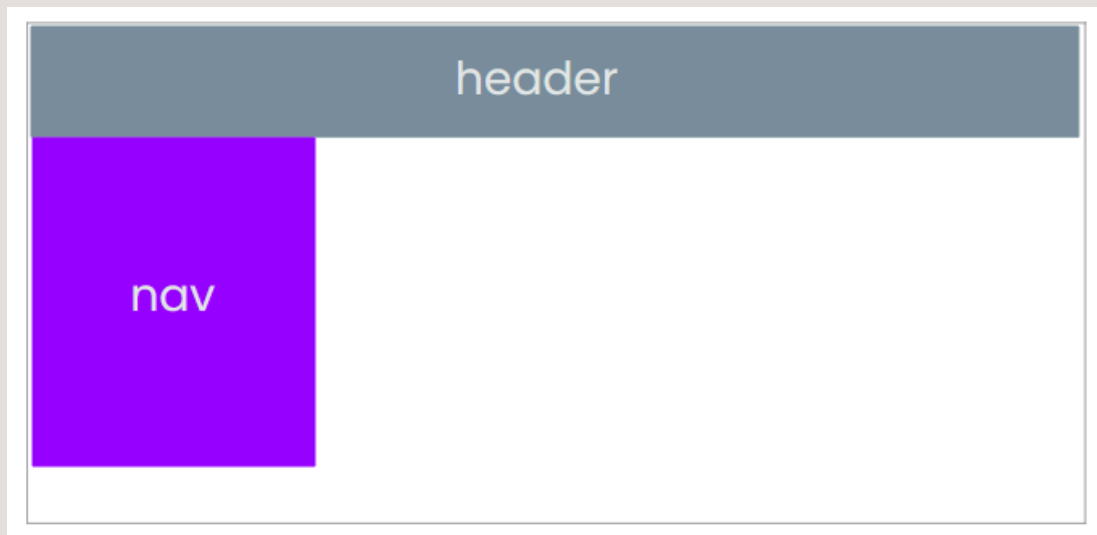
CSS

```
/* ... */
nav {
  background-color: blueviolet;
  grid-row: 2;
  grid-column: 1;
}
```





Resultado:



Tá ficando bacana! Vamos agora criar o conteúdo do site utilizando o elemento main. Vamos indicar pelo CSS que o main (nosso conteúdo principal) vai estar localizado na segunda linha do grid e na segunda coluna.

HTML

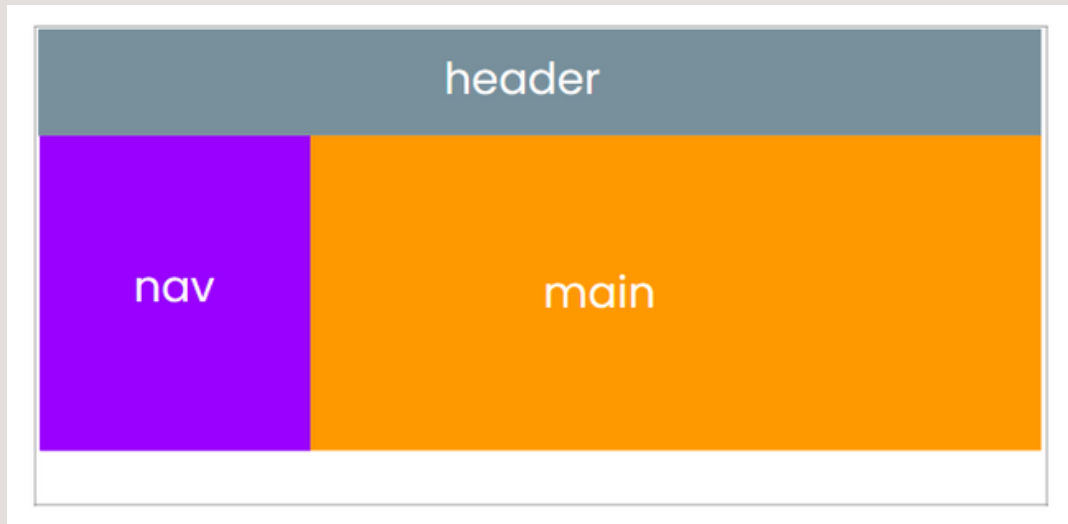
```
<section class="container">
  <header></header>
  <nav></nav>
  <main></main>
</section>
```

CSS

```
/* ... */
main {
  background-color: orange;
  grid-row: 2;
  grid-column: 2;
```



Resultado:



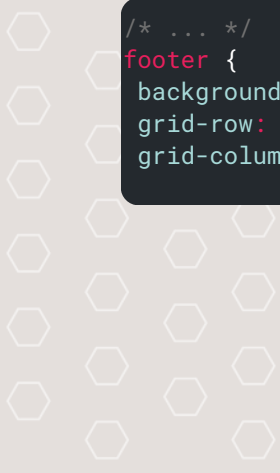
Por último, vamos criar um rodapé para finalizar o nosso grid. Ele vai seguir a mesma lógica do header, a única coisa que vai mudar é a linha que ele está posicionado.

HTML

```
<section class="container">
  <header></header>
  <nav></nav>
  <main></main>
  <footer></footer>
</section>
```

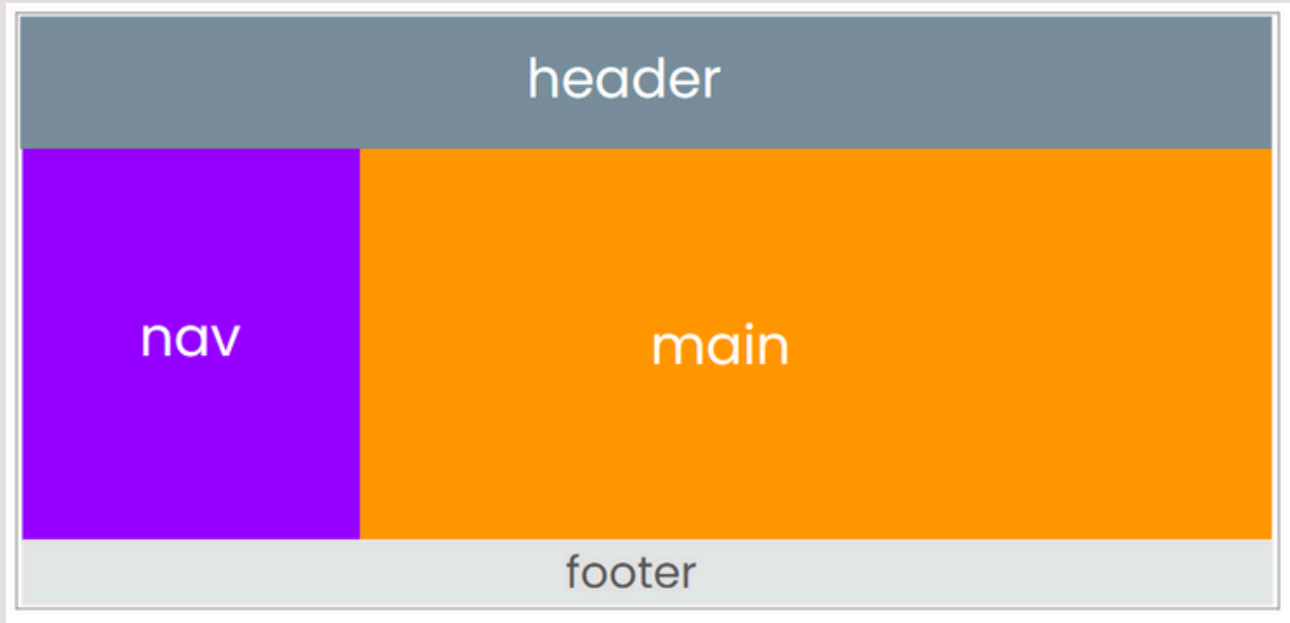
CSS

```
/* ... */
footer {
  background-color: grey;
  grid-row: 3;
  grid-column: 1/3;
```





Resultado:



GRID GAP

A propriedade `grid-gap` é usada para adicionar espaçamentos entre linhas e colunas. Ela deve ser adicionada no container do grid.

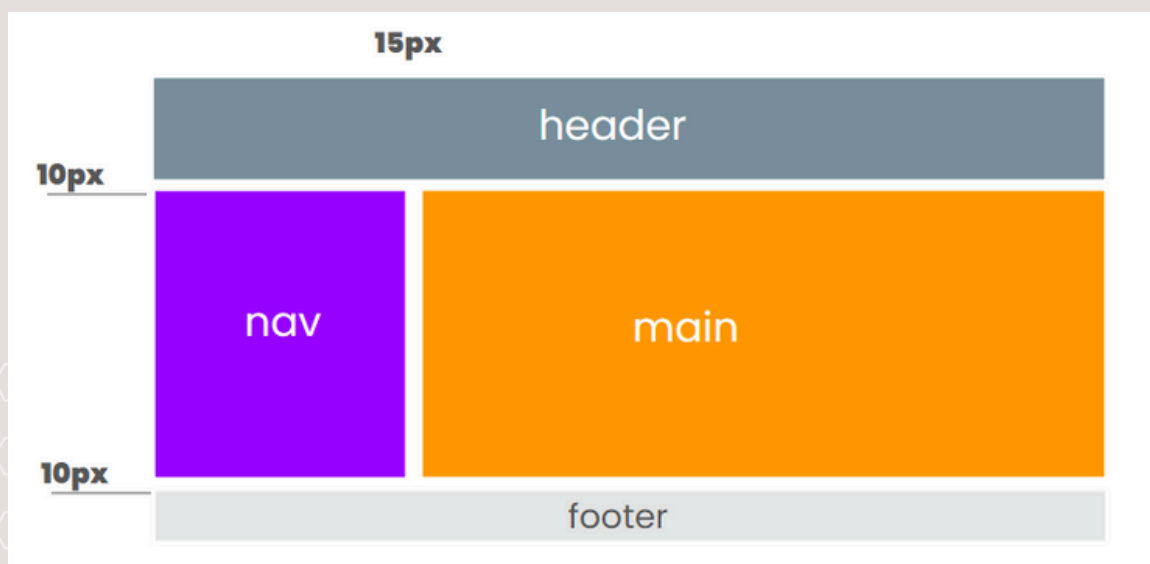
O código comentado simboliza uma outra maneira de definir o gap entre linhas e colunas. Utilizando da forma simplificada (a forma que optamos no exemplo) o primeiro espaçamento a ser definido é o das linhas e o segundo os das colunas. Mas, como o código comentado indica, você pode definir os valores de espaçamento de colunas e linhas separadamente.



CSS

```
/* ... */
.container {
  display: grid;
  grid-template-rows: 100px auto 50px;
  grid-template-columns: 300px auto;
  grid-gap: 10px 15px; /* código adicionado */
  /*
    grid-row-gap: 10px;
    grid-column-gap: 15px;
  */
  width: 100vw;
  height: 100vh;
}
```

Resultado:





GRID AREA

Com essa propriedade é possível dar um “apelido” para a sua sessão, sendo desnecessário utilizar as propriedades `grid-row` e `grid-column`.

No código abaixo, substituímos as propriedades `grid-row` e `grid-column` por `grid-area` com o “apelido” de cada sessão do nosso site.

CSS

```
/* ... */
header {
  background-color: cadetblue;
  grid-area: header;
}

nav {
  background-color: blueviolet;
  grid-area: menu;
}
```





```
main {  
  background-color: orange;  
  grid-area: content;  
}  
  
footer {  
  background-color: grey;  
  grid-area: footer;  
}
```

Bem, com essa modificação surge a questão: “Tendo apagado em quais linhas e colunas o header vai ficar localizado, como é que isso vai funcionar?”

É aí que entra a propriedade `grid-template-areas` que vamos definir no container. Agora é só passarmos o apelido que criamos em suas respectivas linhas/colunas.

CSS

```
/* ... */  
  
.container {  
  width: 100vw;  
  height: 100vh;
```





```
display: grid;
grid-template-rows: 100px auto 50px;
grid-template-columns: 300px auto;
grid-gap: 10px 15px;
grid-template-areas: "header header"
                    "menu content"
                    "footer footer"; /* código adicionado */
}
```

Ou seja:

- **header** na primeira linha, colunas 1 e 2.
- **menu** na segunda linha, colunas 1.
- **content** na segunda linha, coluna 2.
- **footer** na 3 linha, colunas 1 e 2.

Mas e agora, devemos abandonar o Flexbox e começar a usar Grid Layout? Não, de forma alguma! Esses dois módulos CSS devem ser usados juntos pois um complementa o outro. Um exemplo interessante de se considerar é: CSS Grid Layout é uma casa, enquanto o Flexbox são os móveis desta casa.

Para praticar CSS Grid Layout e aplicá-lo em seus sites, recomendamos um projeto bem bacana que te auxilia a criar esses grids:

- [CSS Grid Generator](#)





REFERÊNCIAS

- [CSS Grid Layout Guide | CSS-Tricks](#)
- [CSS Grid Layout \(w3schools.com\)](#)
- [Conceitos básicos de Grid Layout - CSS | MDN \(mozilla.org\)](#)



BONS ESTUDOS!

